

MI DSP API

Version 3.2

© 2021 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

REVISION HISTORY

Revision No.	Description	Date
3.0	Initial release	03/18/2021
	Added procfs introduction	08/25/2021
3.1	Modify the MI_DSP_FW_PACKING_TYPE_e enumeration variable	12/02/2021
3.2	Add Asynchronous Invocation APIs	12/15/2021

TABLE OF CONTENTS

REVISION HISTORY	i
TABLE OF CONTENTS	ii
1. OVERVIEW	1
1.1. Module description	1
1.2. Keyword	1
2. API REFERENCE	3
2.1. MI_DSP_CreateDev	3
2.2. MI_DSP_DestroyDev	4
2.3. MI_DSP_Invoke	4
2.4. MI_DSP_InvokeAsync	6
2.5. MI_DSP_WaitDone	8
3. DATA TYPE	11
3.1. MI_DSP_DEV_e	11
3.2. MI_DSP_DevAttr_t	12
3.3. CustReadFunc	13
3.4. MI_DSP_BufVector_t	13
3.5. MI_DSP_BufInfo_t	14
3.6. MI_DSP_PARAM_t	14
3.7. MI_DSP_PRI_e	15
3.8. MI_DSP_FW_PACKING_TYPE_e	16
4. ERROR CODE	17
5. EXAMPLE CODE	18
6. PROCFS INTRODUCTION	21
6.1. Debug firmware with GDB	21
6.1.1 Summary	21
6.1.2 JTAG interface switch	21
6.1.3 xt-ocd tool installation	21
6.1.4 xt-ocd tool operation	21
6.1.5 xt-ocd configuration file	22
6.1.6 GDB debugging	23
6.2. Debug firmware with xplorer IDE	25
6.2.1 Summary	25
6.2.2 IDE debugging	25

1. OVERVIEW

1.1. Module description

MI DSP is the DSP core embedded in the SStar platform, used to accelerate the development of intelligent vision. MI provides APIs to facilitate rapid development and in-depth customization of users to improve product competitiveness.

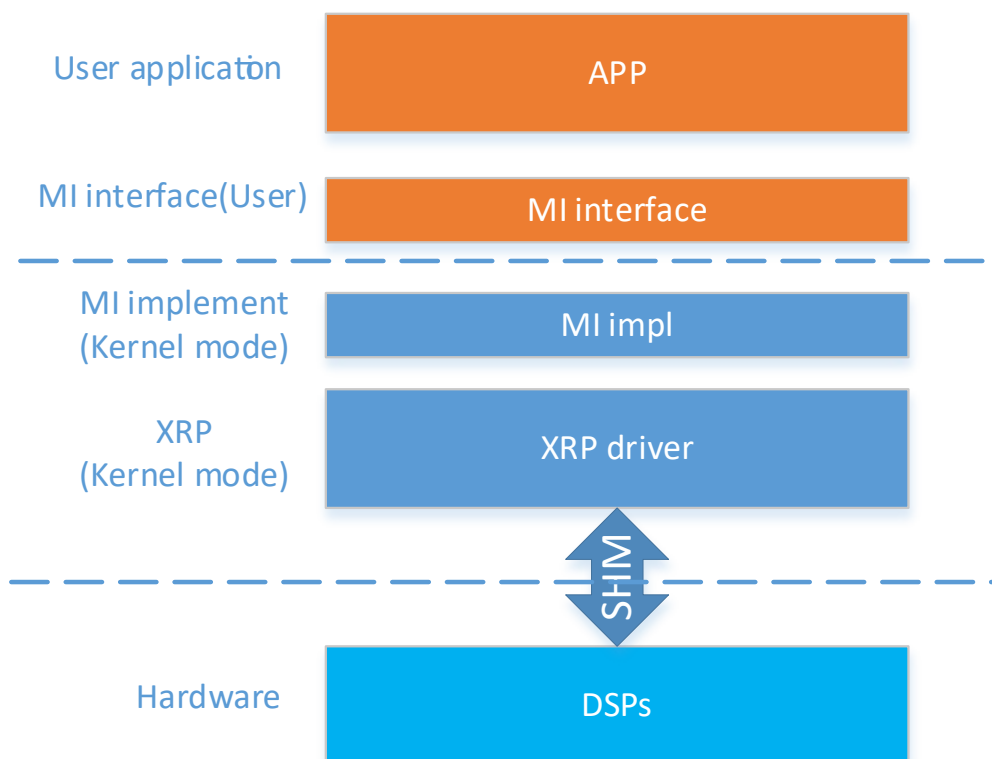


Figure 1-1 MI DSP Development Framework

As Figure 1-1 shown above, MI DSP interface mainly provides API for user mode application. Starting from User application, call MI DSP interface to MI impl in kernel mode, then to driver in kernel mode, and finally interact with DSP through share memory.

1.2. Keyword

ID: Identity document, which represents a unique code.

MI: SStar SDK Middle Interface.

Hex: Hexadecimal.

Kernel Mode: Code working in Kernel, which has the authority to operate the hardware directly. Such as functions and threads in ko.

User Mode: Code working in User, such as user applications, system calls, etc.

XRP: Remote Processing, Linux-based communication interface, used to interact with DSP.

APP: Application, refers to applications that call MI API.

API: Application Programming Interface.

IMPL: Implement, the realization of MI in kernel.

SHM: Share memory.

IPC: IP Camera.

HW: hardware.

2. API REFERENCE

The MI DSP interface provided to the user application is as follows.

API	Fuction
System function	
MI_DSP_CreateDev	Create DSP device
MI_DSP_DestroyDev	Destroy DSP device
MI_DSP_Invoke	Execute the DSP customized algorithm once and get the output

2.1. MI_DSP_CreateDev

➤ Description

Create DSP device.

➤ Syntax

```
MI_S32 MI_DSP_CreateDev(MI\_DSP\_DEV\_e eDspDev, MI\_DSP\_DevAttr\_t *pstDspDevAttr,
CustReadFunc pReadFunc, char *pCtx, MI_U32 fwSize);
```

➤ Parameter

Parameter name	Description	Input/Output
eDspDev	Device ID, corresponding to DSP core ID	Input
pstDspDevAttr	Device attribute	Input
pReadFunc	User-defined function pointer for reading files	Input
pCtx	Firmware file path	Input
fwSize	Firmware file size	Input

➤ Return value

- 0 successful.
- Non-Zero: failed, see [error code](#) for details.

➤ Dependence

- Header: mi_dsp.h
- Library: libmi_dsp.a / libmi_dsp.so

※ Note

When the user uses the custom read file function pReadFunc, fwSize needs to be the same as the actual file size specified by the pCtx path.

When fwSize is 0, pReadFunc should be set to NULL; If the user uses the user-defined read file function pReadFunc, it is an incorrect usage. Please follow the precautions above.

➤ Example

```

MI_S32 s32Ret;
MI_DSP_DevAttr_t stDevAttr;

stDevAttr.eFwPackingType = E_MI_DSP_FW_PACKING_PLAIN;

char *pReadCtx = "/mnt/dsp_firmware";

s32Ret = MI_DSP_CreateDev(MI_DSP_DEV_0, &stDevAttr, NULL, pReadCtx, 0);

if(s32Ret != MI_SUCCESS){
    printf("Fail to create DSP device \n");
    return s32Ret;
}

```

2.2. MI_DSP_DestroyDev

➤ Description

Destroy DSP device.

➤ Syntax

MI_S32 MI_DSP_DestroyDev([MI_DSP_DEV_e](#) eDspDev);

➤ Parameter

Parameter name	Description	Input/Output
eDspDev	Device ID, corresponding to DSP core ID	Input

➤ Return value

- 0 successful.
- Non-Zero: failed, see [error code](#) for details.

➤ Dependence

- Header: mi_dsp.h
- Library: libmi_dsp.a / libmi_dsp.so

※ Note

- u32Dev must be the DSP device created before.

2.3. MI_DSP_Invoke

➤ Description

Execute the DSP customized algorithm once.

➤ Syntax

```
MI_S32 MI_DSP_Invoke(MI\_DSP\_DEV\_e eDspDev, MI\_DSP\_BufVector\_t *pstInputBuf,
MI_DSP_BufVector_t *pstOutputBuf, MI\_DSP\_PARAM\_t *pstParam, MI\_DSP\_PRI\_e ePri, MI_U16
u16TimeOutMs);
```

➤ Parameter

Parameter name	Description	Input/Output
eDspDev	Device ID, corresponding to DSP core ID	Input
pstInputBuf	InputDSP array structure pointer	Input
pstOutputBuf	DSPOutput array structure pointer	Input
pstParam	InputDSP array structure pointer	Input/Output
ePri	The priority of this Invoke	Input
u16TimeOutMs	The timeout period of this Invoke	Input

➤ Return value

- 0 successful.
- Non-Zero: failed, see [error code](#) for details.

➤ Dependence

- Header: mi_dsp.h
- Library: libmi_dsp.a / libmi_dsp.so

※ Note

- [MI_DSP_Invoke](#) is a blocking API, you get a result once you execute it.
- pstParam->s32Result returns the result from DSP side for the invoke.

➤ Example

```
MI_DSP_BufVector_t stInputBuf, stOutputBuf;
MI_DSP_PARAM_t stParam;
MI_S32 retVal = 0;

memset(&stInputBuf, 0, sizeof(MI_DSP_BufVector_t));
memset(&stOutputBuf, 0, sizeof(MI_DSP_BufVector_t));

/* 1. Allocate InputBuf space through MMA and map it */
stInputBuf.astBufInfoVec[0].u32Size = 0x1000;
MI_SYS_MMA_Alloc(0, NULL, stInputBuf.astBufInfoVec[0].u32Size, &stInputBuf.astBufInfoVec[0].phyAddr);
MI_SYS_Mmap(stInputBuf.astBufInfoVec[0].phyAddr, stInputBuf.astBufInfoVec[0].u32Size, &virtAddrIn,
FALSE);
stInputBuf.u32BufCount = 1;
stInputBuf.astBufInfoVec[0].u32Flag = 0x1;
memset(virtAddrIn, 0, stInputBuf.astBufInfoVec[0].u32Size);
```



```

/* 2. Allocate OutputBuf space through MMA and map it */
stOutputBuf.astBufInfoVec[0].u32Size = 0x1000;
MI_SYS_MMA_Alloc(0, NULL, stOutputBuf.astBufInfoVec[0].u32Size, &stOutputBuf.astBufInfoVec[0].phyAddr);
MI_SYS_Mmap(stOutputBuf.astBufInfoVec[0].phyAddr, stOutputBuf.astBufInfoVec[0].u32Size, &virtAddrOut,
FALSE);
stOutputBuf.u32BufCount = 1;
stOutputBuf.astBufInfoVec[0].u32Flag = 0x2;
stParam.u64Data[0] = 0x2222;
memset(virtAddrOut, 0, stOutputBuf.astBufInfoVec[0].u32Size);

/* 3. Assign a value to InputBuf */
for(int i = 0; i < stInputBuf.astBufInfoVec[0].u32Size; i++) {
    char* p = (char*)virtAddrIn;
    p[i] = 0xc2;
}

/* 4. Call the algorithm of dsp core0 once */
retVal = MI_DSP_Invoke(0, &stInputBuf, &stOutputBuf, &stParam, 0, 65535);
if (retVal != MI_SUCCESS)
{
    printf("core 0 invoke err, retVal: 0x%x!\n", retVal);
    goto invoke_err;
}

/* 5. Print processing results */
for(int i = 0; i < 10; i++) {
    char* p = (char*)virtAddrOut;
    printf("%d \n", p[i]);
}

```

2.4. MI_DSP_InvokeAsync

- Description
Commit one computation request to DSP asynchronously.
- Syntax
MI_S32 MI_DSP_InvokeAsync([MI_DSP_DEV_e](#) eDspDev, [MI_DSP_BufVector_t](#) *pstInputBuf, [MI_DSP_BufVector_t](#) *pstOutputBuf, [MI_DSP_PARAM_t](#) *pstParam, [MI_DSP_PRI_e](#) ePri, MI_U32 *u32fence);
- Parameter

Parameter name	Description	Input/Output
eDspDev	Device ID, corresponding to DSP core ID	Input
pstInputBuf	InputDSP array structure pointer	Input
pstOutputBuf	DSPOutput array structure pointer	Input
pstParam	InputDSP array structure pointer	Input/Output
ePri	The priority of this Invoke	Input
u32fence	Request ID of this Invoke to assign	Output

➤ Return value

- 0 successful.
- Non-Zero: failed, see [error code](#) for details.

➤ Dependence

- Header: mi_dsp.h
- Library: libmi_dsp.a / libmi_dsp.so

※ Note

- [MI_DSP_InvokeAsync](#) is a non-blocking API that returns immediately after execution. Other than MI_SUCCESS, A typical error MI_DSP_ERR_PRIQ_FULL will be given. It implies that DSP is busy handling requests in request queue which was fully queued at the moment. In such case, function [MI_DSP_WaitDone](#) was introduced to wait for certain request complete, then DSP will be available for new Invoke commit again.
- Parameter 'u32fence' will be assigned with request ID if function return success. Function [MI_DSP_WaitDone](#) can be used to wait for the request complete with it.
- Different from [MI_DSP_Invoke](#), pstParam->s32Result will not be assigned after the function call. In order to get the result of the DSP side for this Invoke, please refer to function [MI_DSP_WaitDone](#).

➤ Example

InputBuf and OutputBuf were allocated with APIs from MI_SYS module.

```
MI_U32 lastfence = 0; int tmp = 5;
while (tmp)
{
    for (int i = 0; i < stInputBuf.astBufInfoVec[0].u32Size; i++)
    {
        char* p = (char*)virtAddrIn;
        p[i] = tmp;
    }

    MI_U32 fence = 0;
    while(MI_SUCCESS != MI_DSP_InvokeAsync(0, &stInputBuf, &stOutputBuf, &stParam, 0, &fence))
```

```

{
    printf("dev busy %d \n",tmp);
    //usleep(19);
    if(MI_DSP_WaitDone(0,0, lastfence, NULL, 20000)!=MI_SUCCESS)
    {
        printf("dev busy %d timeout\n",tmp);
    }
}
lastfence = fence;
//MI_DSP_Invoke(0, &stInputBuf, &stOutputBuf, &stParam, 0, 100);
// if(tmp == 3)
//    MI_DSP_ResetDev(0);
for (int i = 0; i < 10; i++)
{
    char* p = (char*)virtAddrOut;
    printf("%d \n", p[i]);
}
printf("fence:%d\n", fence);
tmp--;
}

```

2.5. MI_DSP_WaitDone

➤ Description

Wait for an asynchronous DSP([MI_DSP_InvokeAsync](#)) computation request complete.

➤ Syntax

MI_S32 MI_DSP_WaitDone(MI_DSP_DEV_e eDspDev, MI_DSP_PRI_e ePri, MI_U32 u32fence, MI_S32* s32Result, MI_U32 u32TimeOutMs);

➤ Parameter

Parameter name	Description	Input/Output
eDspDev	Device ID, corresponding to DSP core ID	Input
ePri	The priority of this Invoke	Input
u32fence	Request ID to wait	Input
s32Result	Result of the latest finished task fence, maybe not corresponding to the parameter: u32fence	Output
u32TimeOutMs	Max time period to wait	Input

➤ Return value

- 0 successful.
- Non-Zero: failed, see [error code](#) for details.

➤ Dependence

- Header: mi_dsp.h
- Library: libmi_dsp.a / libmi_dsp.so

※ Note

- Function call was blocked on each DSP dev
- Return value: MI_SUCCESS or MI_DSP_ERR_TIMEOUT
- u32Fence is correspond to eDspDev and ePri , user can wait any sensible fence ID, don't have to wait for earlier ones first. If the later fence ID has been wait completed , naturally the earlier requests are completed already and the function will return straightly.
- s32Result returns the result of the latest finished task fence, it might owes none relationship to the parameter [u32fence]. Eg: u32fence could be a very old task fence which was done way before.

➤ Example

InputBuf and OutputBuf were allocated with APIs from MI_SYS module.

```
MI_U32 lastfence = 0; int tmp = 5;
while (tmp)
{
    for (int i = 0; i < stInputBuf.astBufInfoVec[0].u32Size; i++)
    {
        char* p = (char*)virtAddrIn;
        p[i] = tmp;
    }

    MI_U32 fence = 0;
    while(MI_SUCCESS != MI_DSP_InvokeAsync(0, &stInputBuf, &stOutputBuf, &stParam, 0, &fence))
    {
        printf("dev busy %d \n",tmp);
        //usleep(19);
        if(MI_DSP_WaitDone(0,0, lastfence, NULL, 20000)!=MI_SUCCESS)
        {
            printf("dev busy %d timeout\n",tmp);
        }
    }
    lastfence = fence;
    //MI_DSP_Invoke(0, &stInputBuf, &stOutputBuf, &stParam, 0, 100);
    // if(tmp == 3)
    //     MI_DSP_ResetDev(0);
    for (int i = 0; i < 10; i++)
    {
        char* p = (char*)virtAddrOut;
        printf("%d \n", p[i]);
    }
    printf("fence:%d\n", fence);
    tmp--;
}
```

}

3. DATA TYPE

The related data types and data structure are defined as follows.

Data type	Definition
MI_DSP_DEV_e	Define the device type
MI_DSP_DevAttr_t	Define device hardware attributes
CustReadFunc	It is used for customized file reading methods, such as encryption, etc.
MI_DSP_BufVector_t	Define multiple Input/OutputBuffer structures
MI_DSP_BufInfo_t	Define the structure of the code stream information.
MI_DSP_PARAM_t	Define customized user parameters, such as immediate data or address/length
MI_DSP_PRI_e	Priority of Invoke API
MI_DSP_FW_PACKING_TYPE_e	Define the packaging type of DSP firmware

3.1. MI_DSP_DEV_e

- Description
Define the device type.

- Definition
typedef enum
{
 MI_DSP_DEV_0 = 0,
 MI_DSP_DEV_1,
 MI_DSP_DEV_2,
 MI_DSP_DEV_3,
 MI_DSP_DEV_BUTT
} MI_DSP_DEV_e;

- Member

Member name	Description
MI_DSP_DEV_0	DSP device Core 0
MI_DSP_DEV_1	DSP device Core 1
MI_DSP_DEV_2	DSP device Core 2
MI_DSP_DEV_3	DSP device Core 3

※ Note
N/A

➤ Related data type and interface
N/A

3.2. MI_DSP_DevAttr_t

➤ Description
Define device hardware attributes

➤ Definition

```
typedef struct MI_DSP_DevAttr_s
{
    MI_DSP_FW_PACKING_TYPE_e  eFwPackingType;
    MI_U32                     u32ResetVectorAddr;
    MI_U16                     u16Prid;
    MI_BOOL                    bRebootFw;
} MI_DSP_DevAttr_t;
```

➤ Member

Member name	Description
eFwPackingType;	Packaging method of DSP firmware
u32ResetVectorAddr	Firmware reset vector address
u16Prid	The processor id specified by the DSP device. Need to be set to 0x272 when debugging.
bRebootFw	Whether to automatically reset the firmware when Invoke times out

※ Note

When the External option is selected in IDE create Custom LSPs to customize the address, u32ResetVecAddr must be consistent with the address, and the value must be 64 aligned; when the Primary option is selected, u32ResetVecAddr must be set to 0.

When some functions in firmware are stored in IRAM, eFwPackingType needs to be set to E_MI_DSP_FW_PACKING_ROMING, so it is necessary to modify and compile the LSP of firmware - memmap.xmm file, add ROMING=true and specs file, and replace the original -lhandler-reset with -lhandler-reset-unpack. For details, see LSP Manual Chapter 3 placing applications in ROM; If no function is stored in IRAM, efwpackingtype is not set to E_MI_DSP_FW_PAKCING_PLAIN.

➤ Related data type and interface
N/A

3.3. CustReadFunc

➤ Description

It is used for customized file reading. Noted that the return value should be greater than or equal to 0 when the function is executed successfully

➤ Definition

typedef int (* CustReadFunc)(void *dst_buf, int offset, int size, char *ctx)

➤ Member

Member name	Description
dst_buf	Data storage address
offset	File pointer offset
size	Read file size
ctx	File path

※ Note

- This function pointer can be set when user needs to customize to read file (such as encryption).
- The dst space specified by this function is allocated internally by the MI_DSP_CreateDev() function, the size is fwSize, and the user does not need to allocate it.
- When the user sets the function to NULL, the default read firmware function is used. The default read function does not need to be provided by the user

➤ Related data type and interface

N/A

3.4. MI_DSP_BufVector_t

➤ Description

Define multiple Input/OutputBuffer structures.

➤ Definition

```
typedef struct MI_DSP_BufVector_s
{
    MI_U32 u32BufCount;
    MI\_DSP\_BufInfo\_t astBufInfoVec[MAX_BUF_CNT];
} MI_DSP_BufVector_t;
```

➤ Member

Member name	Description
u32BufCount	Number of BufInfo
astBufInfoVec	Array of BufInfo

※ Note

The recommended value of MAX_BUF_CNT is 10, and the macro is defined in mi_dsp_datatype.h: #define MAX_BUF_CNT 10.

- Related data type and interface
N/A

3.5. MI_DSP_BufInfo_t

- Description
Define the structure of the code stream information.

- Definition


```
typedef struct MI_DSP_BufInfo_s
{
    MI_PHY phyAddr;
    MI_U32 u32Size;
    MI_U32 u32Flag;
} MI_DSP_BufInfo_t;
```

- Member

Member name	Description
phyAddr	The physical address of the memory passed to DSP
u32Size	The size of the memory passed to DSP
u32Flag	Flag information of the memory passed to DSP

※ Note

Please get continuous physical memory through the MI_SYS interface.

- Related data type and interface
N/A

3.6. MI_DSP_PARAM_t

- Description
Define customized user parameters, such as immediate data or address/length.

- Definition


```
typedef struct MI_DSP_PARAM_s
{
    MI_U64 u64Data[MAX_PARAM_CNT];
    MI_S32 s32Result;
```



```
} MI_DSP_PARAM_t;
```

➤ Member

Member name	Description
u64Data	Custom array passed to DSP's message handler
s32Result	Store the result of DSP side for MI_DSP_Invoke

※ Note

- The recommended value of MAX_PARA_CNT is 2, and the macro is defined in mi_dsp_datatype.h:
#define MAX_PARA_CNT 2.
- When is calling MI_DSP_Invoke, the array can store customized user parameters. If the address is stored, it must be the physical address of the buffer allocated by MI_SYS_MMA_ALLOC, which is convenient for DSP to view.

➤ Related data type and interface

N/A

3.7. MI_DSP_PRI_e

➤ Description

Define the priority of [MI_DSP_Invoke](#), which indicates the priority of MSG queue processed by DSP.

➤ Definition

```
typedef enum
{
    MI_DSP_PRI_0 = 0,
    MI_DSP_PRI_1,
    MI_DSP_PRI_2,
    MI_DSP_PRI_BUTT
} MI_DSP_PRI_e;
```

➤ Member

Member name	Description
MI_DSP_PRI_0	Priority queue level 0, which is the highest
MI_DSP_PRI_1	Priority queue level 1
MI_DSP_PRI_2	Priority queue level 2, which is the lowest

※ Note

N/A

➤ Related data type and interface

N/A

3.8. MI_DSP_FW_PACKING_TYPE_e

➤ Description

Define the packaging type of DSP firmware.

➤ Definition

typedef enum

{

 E_MI_DSP_FW_PACKING_ROMING = 0,

 E_MI_DSP_FW_PACKING_PLAIN,

 E_MI_DSP_FW_PACKING_BUTT

} MI_DSP_FW_PACKING_TYPE_e;

➤ Member

Member name	Description
E_MI_DSP_FW_PACKING_ROMING	Firmware packaging type is ROMING
E_MI_DSP_FW_PACKING_PLAIN	Firmware packaging type is PLAIN

※ Note

When some functions in firmware are stored in IRAM, eFwPackingType needs to be set to E_MI_DSP_FW_PACKING_ROMING, so it is necessary to modify and compile the LSP of firmware - memmap.xmm file, add ROMING=true and specs file, and replace the original -lhandler-reset with -lhandler-reset-unpack. For details, see LSP Manual Chapter 3 placing applications in ROM; If no function is stored in IRAM, efwpackingtype is not set to E_MI_DSP_FW_PAKCING_PLAIN.

➤ Related data type and interface

N/A

4. ERROR CODE

The error codes of the MI DSP module are shown in the following table.

Error code	Macro definition	Description
0xA0242080	MI_DSP_ERR_READ_FILE	Error reading file
0xA0242081	MI_DSP_ERR_INVALID_DEVID	The device number is wrong or out of the hardware support range
0xA0242082	MI_DSP_ERR_DEV_ATTR	Device attribute parameter error
0xA0242083	MI_DSP_ERR_RE_CREATE	Repeat device creation
0xA0242084	MI_DSP_ERR_NULL_PTR	There are pointers that are not allowed to be null in the function parameters
0xA0242085	MI_DSP_ERR_PRIQ_FULL	Priority Queue full

Error code	Macro definition	Description
0xA0242086	MI_DSP_ERR_NOT_READY	The system is not initialized or the firmware is not ready
0xA0242087	MI_DSP_ERR_TIMEOUT	System exception caused timeout
0xA0242088	MI_DSP_ERR_INVALID_PARAM	The parameter is out of the legal range
0xA0242089	MI_DSP_ERR_BAD_ADDRESS	Illegal address
0xA024208A	MI_DSP_ERR_FAILED	Other errors

5. EXAMPLE CODE

```

#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <string.h>

#include <mi_dsp.h>
#include <mi_dsp_datatype.h>
#include <mi_sys.h>

int main()
{
    MI_S32 retVal = 0;
    MI_DSP_DevAttr_t stDspDevAttr;
    MI_DSP_BufVector_t stInputBuf, stOutputBuf;
    MI_DSP_PARAM_t stParam;

    void* virtAddrIn = 0;
    void* virtAddrOut = 0;
    stDspDevAttr.eFwPackingType = E_MI_DSP_FW_PACKING_PLAIN;
    stDspDevAttr.u32ResetVecAddr = 0x10040000;
    stDspDevAttr.u16Prid = 0x272;

    /* 1. Create a dsp core0 device and specify the firmware path */
    retVal = MI_DSP_CreateDev(0, &stDspDevAttr, NULL,
        "/lib/firmware/tc_invoke.elf", 0);
    if (retVal != MI_SUCCESS)
    {
        printf("core 0 createdev err, retVal: 0x%x!\n", retVal);
        goto create_err;
    }
}

```

```

memset(&stInputBuf, 0, sizeof(MI_DSP_BufVector_t));
memset(&stOutputBuf, 0, sizeof(MI_DSP_BufVector_t));

/* 2. Allocate InputBuf space through MMA and map it */
stInputBuf.astBufInfoVec[0].u32Size = 0x1000;
MI_SYS_MMA_Alloc(0, NULL, stInputBuf.astBufInfoVec[0].u32Size,
    &stInputBuf.astBufInfoVec[0].phyAddr);
MI_SYS_Mmap(stInputBuf.astBufInfoVec[0].phyAddr,
    stInputBuf.astBufInfoVec[0].u32Size, &virtAddrIn, FALSE);
stInputBuf.u32BufCount = 1;
stInputBuf.astBufInfoVec[0].u32Flag = 0x1;
memset(virtAddrIn, 0, stInputBuf.astBufInfoVec[0].u32Size);

/* 3. Allocate OutputBuf space through MMA and map it */
stOutputBuf.astBufInfoVec[0].u32Size = 0x1000;
MI_SYS_MMA_Alloc(0, NULL, stOutputBuf.astBufInfoVec[0].u32Size,
    &stOutputBuf.astBufInfoVec[0].phyAddr);
MI_SYS_Mmap(stOutputBuf.astBufInfoVec[0].phyAddr,
    stOutputBuf.astBufInfoVec[0].u32Size, &virtAddrOut, FALSE);
stOutputBuf.u32BufCount = 1;
stOutputBuf.astBufInfoVec[0].u32Flag = 0x2;
stParam.u64Data[0] = 0x2222;
memset(virtAddrOut, 0, stOutputBuf.astBufInfoVec[0].u32Size);

/* 4. Assign a value to InputBuf */
for(int i = 0; i < stInputBuf.astBufInfoVec[0].u32Size; i++) {
    char* p = (char*)virtAddrIn;
    p[i] = 0xc2;
}

/* 5. Call the algorithm of dsp core0 once */
retVal = MI_DSP_Invoke(0, &stInputBuf, &stOutputBuf, &stParam, 0,
65535);
if (retVal != MI_SUCCESS)
{
    printf("core 0 invoke err, retVal: 0x%x!\n", retVal);
    goto invoke_err;
}

/* 6. Print processing results */
for(int i = 0; i < 10; i++) {
    char* p = (char*)virtAddrOut;
    printf("%d \n", p[i]);
}

```

```

        /* 7. Carry out resource release work */
invoke_err:
    MI_SYS_Munmap(virtAddr, 0x1000);
    MI_SYS_MMA_Free(0, u64PhyAddr);
    MI_DSP_DestroyDev(0);

create_err:
    while(1) {
        sleep(1);
    }

    return 0;
}

```

6. PROCFS INTRODUCTION

6.1. Debug firmware with GDB

6.1.1 Summary

Configure the software system correctly and run the MI_DSP APP on the board.

Configure the Jlink environment on the debug host and connect the JTAG usb debug tool correctly.

6.1.2 JTAG interface switch

In the SOC circuit on the default board side, JTAG is connected to ARM cores. You need to type the following commands in the console of the board to connect JTAG to the DSP cores.

- /customer/riu_w 101e 7b 101; /customer/riu_w 101e 7b 001; When JTAG switches to DSP cores, xt-ocd can be connected.
- /customer/riu_w 101e 7b 0; JTAG switches back to ARM cores.

6.1.3 xt-ocd tool installation

Download xt-ocd-14.05-windows64-installer.exe through xplorer XPG. If the debug host is in a Linux environment, please download the Linux version and install it.

Running xt-ocd requires jlink library: Jlink_x64.dll (windows), it is recommended to copy it to the modules folder of the xt-ocd installation directory.

6.1.4 xt-ocd tool operation

For details about how xt-ocd works, see the cadence official document: Xtensa® Debug Guide.

Xt-ocd operation needs to be performed after MI_DSP createdev, before that DSP cores have not been initialized.

Basic operation mode: Run xt-ocd.exe -c mytopology.xml on the debug host. Among them mytopology.xml is the configuration file.

After successful operation, if there is a log similar to the following, the IR bits and width of JTAG can be

obtained correctly.

```
D:\Program Files (x86)\Tensilica\Xtensa OCD Daemon 14.05>xt-ocd.exe -c mytopology.xml
XOCD 14.05 2020-10-12 17:38:02
(c) 1999-2021 Cadence Design Systems Inc. All rights reserved.
[Debug Log 2021-03-04 16:54:53]
Loading module "gdbstub" v2.0.0.12
Loading module "jlink" v2.0.2.0
Using JLINK lib v.61210
Jlink USB Serial Number: 261003840
Connected to Jlink Device:
  Name:'J-Link V11 compiled Aug 14 2019 16:21:09'
  S/N:261003840
  Firmware: J-Link V11 compiled Aug 14 2019 16:21:09
  Requested/Set TCK: 2000kHz/2000kHz
Loading module "jtag" v2.0.0.20
Loading module "xtensa" v2.0.0.48
Loading module "traxapp" v2.0.0.8
Loading module "trax" v2.0.1.23
Starting thread 'GDBStub'
Starting thread 'TraxApp'
Opened GDB socket at port 20000
traxapp: listening on port 11444
Initialize XDM driver
Total IR bits : 5
TAP[0] irwidth = 5
```

6.1.5 xt-ocd configuration file

```
<configuration>
  <controller id='Controller0' module='jlink' usbser='261008694' speed='10000000' />
  <!--
    Default Flyswatter2
  <controller id='Controller0' module='ft2232' probe='flyswatter2' speed='10MHz' />

  Here are other example controller (probe) lines, that can replace the above one
  when using different JTAG probes (scan controllers):

  Cadence Design Systems ML605 Daughterboard - optional usbser is inventory sticker number prefixed with 'ML605-':
  <controller id='Controller0' module='ft2232' probe='ML605' speed='10MHz' usbser='ML605-2147' />

  Tin Can Tools Flyswatter2:
  <controller id='Controller0' module='ft2232' probe='flyswatter2' speed='10MHz' />

  RVI/DSTREAM: NOTE: Probe settings are in the loaded rvc file
```

```
</configuration>
```

SEGGER J-Link V6.12j - Control panel

General | Settings | Breakpoints | RTT | Log | CPU Regs | Target Power | SWV | RAWTrace

☒ Start minimized
☒ Always on top

Process: D:\Program Files (x86)\Tensilica\Vtensa OCD Daemon 1.v
 DLL: D:\Program Files (x86)\Tensilica\Vtensa OCD Daemon 1.v
 J-Link: J-Link V11 compiled Aug 14 2019 16:21:09 V11.0, SN=2f...
 Device: Not selected Little endian 3.32 V
 Target interface: JTAG 2000 kHz Actual: 2000 kHz
 Host interface: USB (SN 261003640)

License About

Ready JLINK_JTAG_SyncBits (Done) 0.024 sec. in 174 calls

12/15/2021

6.1.6 GDB debugging

1. Run xt-gdb on the local debugging console: `xt-gdb.exe --xtensa-core=T12_DSP_eva HelloWorld1`
2. Connect to the HAPS remote environment 'target remote [host]:[port]':
target remote 172.19.32.104:20000
3. Turn off automatic local connection: `set auto-connect-native-target off`
4. Load program: `load`
5. Continue running program: `continue`

```
+ Attach a running log:
...
E:\usr\xtensa\XtDevTools\install\tools\RI-2020.5-win32\XtensaTools\bin>xt-gdb.exe --xtensa-
core=T12_DSP_fpga HelloWorld1
GNU gdb (GDB) 7.11.1 Xtensa Tools 14.05
Copyright (C) 2016 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law. Type "show copying"
and "show warranty" for details.
This GDB was configured as "--host=x86_64-w64-mingw32 --target=xtensa-elf".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from HelloWorld1...done.
(xt-gdb) target remote 172.19.32.104:20000
Remote debugging using 172.19.32.104:20000
0x00000000 in ?? ()
(xt-gdb) set auto-connect-native-target off
(xt-gdb) load
Loading section .DispatchVector.text, size 0x25c lma 0x10000000
Loading section .DispatchHandler.text, size 0x4 lma 0x1000025c
Loading section .ResetVector.text, size 0x4 lma 0x10004000
Loading section .ResetHandler.text, size 0x18c lma 0x10004004
Loading section .clib.rodata, size 0x618 lma 0x20000000
Loading section .rtos.rodata, size 0x80 lma 0x20000640
Loading section .rodata, size 0x218 lma 0x200006c0
Loading section .text, size 0xa298 lma 0x200008e0
Loading section .clib.data, size 0x1bc lma 0x2000ab80
Loading section .data, size 0x233c lma 0x2000ad40
Start address 0x10004000, load size 54320
Transfer rate: 26 KB/sec, 1597 bytes/write.
(xt-gdb) continue
```



```
Continuing.  
main 34  
main 36 cc  
main 36 cc  
main 36 cc  
main 36 cc  
main 36 cc  
main 36 cc  
main 36 cc  
main 36 cc  
main 36 cc  
main 36 cc  
main 36 cc  
main 36 cc  
hello world 0  
  
Program received signal SIGTRAP, Trace/breakpoint trap.  
_exit () at /build/tree/RI-2020.5_kuma/ib/tools/swtools-MSWin32-x86/xtensa-elf/src/xtos/exit.S:88  
88  /build/tree/RI-2020.5_kuma/ib/tools/swtools-MSWin32-x86/xtensa-elf/src/xtos/exit.S: No such file o  
r directory.  
(xt-gdb)
```

※ ***For the debugging manual of Xt-gdb, please refer to the official document of cadence: GNU Debugger User's Guide***

6.2. Debug firmware with xplorer IDE

6.2.1 Summary

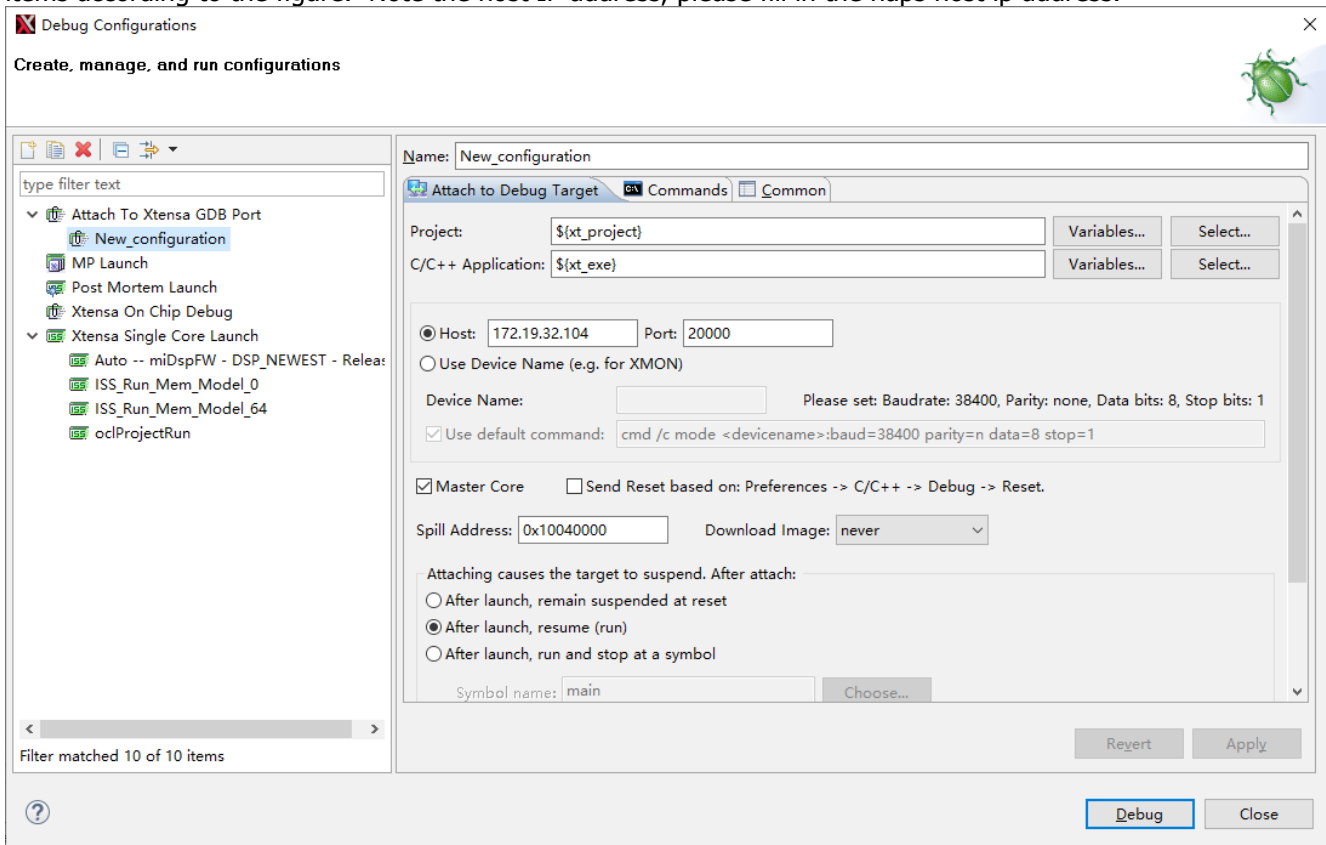
In addition to using the xt-gdb command to debug the firmware, you can also use the xplorer IDE. IDE debugging and gdb direct debugging require the same preparation work, both of which need to perform Section 5.1.1 to 5.1.5 of Chapter 5.1.

6.2.2 IDE debugging

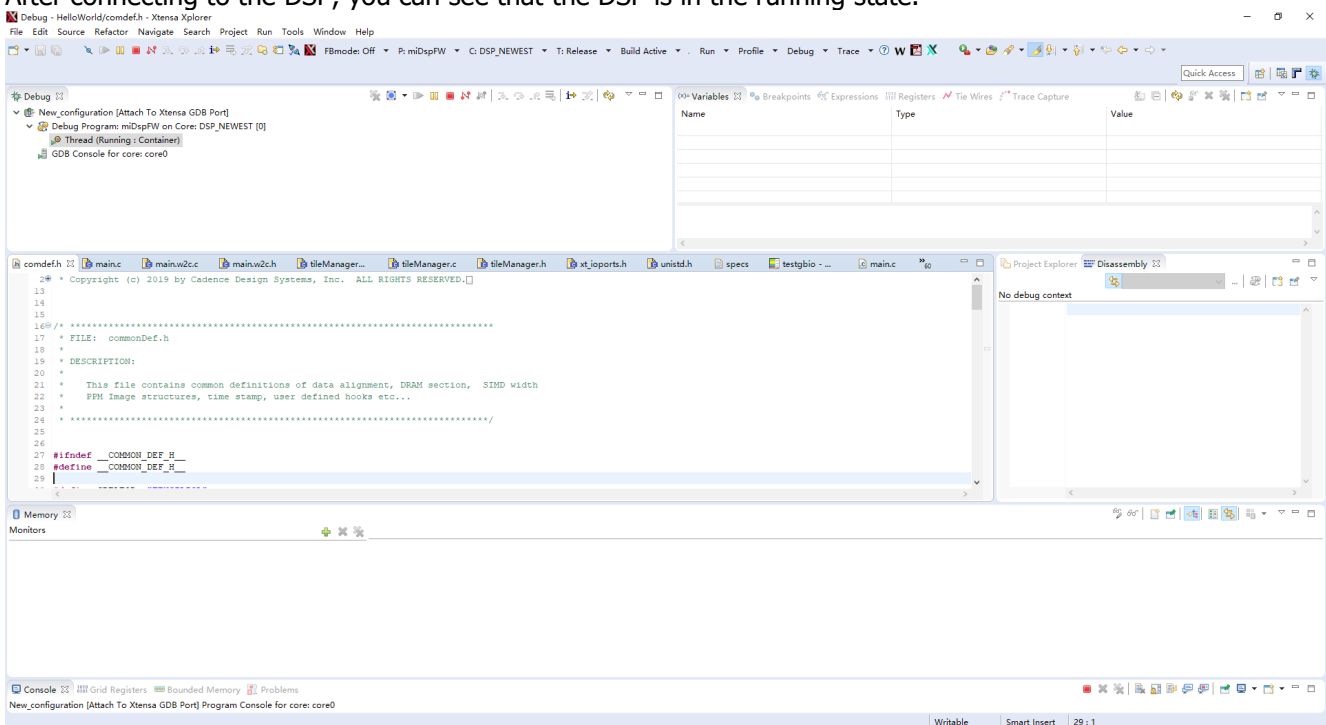
Use xplorer to connect to the dsp, configure the running firmware in the xplorer toolbar, and select debug configuration in the debug option.



As shown in the figure below, create a new attach to xtensa gdb port configuration and select configuration items according to the figure. Note the host IP address, please fill in the haps host ip address.



After connecting to the DSP, you can see that the DSP is in the running state.



Select pause->||, and then select pause. At this time, the DSP will be stalled. At this time, you can view various information of the system through the IDE.

Such as: running stack, memory information, variable information, register information, etc.

